Tech Challenge — Fase 1

Diagnóstico de Câncer de Mama

Modelos Preditivos: Random Forest & KNN

```
# Tech Challenge - Fase 1
```

1. Definição do Problema e Planejamento

O diagnóstico de câncer de mama a partir de biópsias envolve a análise visual de características dos núcleos celulares como tamanho, forma, irregularidade do contorno. Esse processo é trabalhoso, sujeito a variações entre diferentes profissionais e exige especialização. A pergunta que motivou este trabalho foi: é possível ensinar um programa de computador a identificar padrões que separam tumores benignos de malignos com a mesma confiabilidade de um especialista?

Para responder a isso, utilizamos um conjunto de dados com 569 biópsias reais, cada uma descrita por 30 características celulares medidas em laboratório. Desses casos, 357 eram benignos e 212 malignos. Dois modelos foram treinados e comparados, o Random Forest e o KNN.

2. Coleta e Entendimento dos Dados

Informações do dataset

Para treinamento e validação do modelo utilizamos o dataset disponibilizado pela <a href="https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Diagnostic%29">Universidade da Califórnia Irvine (UCI)</a>.

Além do campo ID (identificador da amostra) e nosso target diagnosis (diagnóstico), o dataset possui os campos abaixo:

Característica do núcleo celular	Significado

radius	Média da distância do centro ao perímetro do núcleo
texture	Variação nos níveis de cinza da imagem
perimeter	Comprimento do contorno do núcleo
area	Área total do núcleo
smoothness	Variação local no comprimento do raio
compactness	Relação perímetro² / área – 1
concavity	Severidade das porções côncavas do contorno
concave points	Número de pontos côncavos no contorno
symmetry	Simetria do núcleo
fractal dimension	("aproximação do contorno" - 1)

Cada característica do núcleo celular aparece em três versões:

- **_mean:** é a média dos valores
- **_se:** erro padrão da média (standard error). Demonstra o quão confiável é a média.
- **_worst:** maior valor observado

---

3. Análise dos Dados

In [ ]

```
import numpy as np
import pandas as pd
import seaborn as sns
import shap
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, f1_score, accuracy_score, classification_report, co
from sklearn.neighbors import KNeighborsClassifier
```

In [ ]

```
# Lendo o arquivo do dataset e criando o dataframe
df = pd.read_csv("data.csv" , sep=",")
```

In [ ]

```
# Verificando a dimensão do dataframe
print(df.shape[0], "linhas")
print(df.shape[1], "colunas")
```

```
569 linhas
33 colunas
```

In [ ]

```
# Exibindo detalhes estruturais do dataframe
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtype
```

Analisando as colunas constatamos que:

- A única coluna descritiva é o nosso target diagnosis
- Quase todos os campos possuem 569 ocorrências de dados non-null, o que já nos indica que não há colunas nulas, exceto a coluna Unnamed.

O campo Unnamed possui todas as linhas nulas o que indica ser uma má formação no nosso dataset. Sendo assim podemos excluí-lo.

In [ ]

```
# Verificando se tem algum campo nulo
df.isnull().sum()
```

```
id                           0
diagnosis                    0
radius_mean                  0
texture_mean                 0
perimeter_mean               0
area_mean                    0
smoothness_mean              0
compactness_mean             0
```

```
concavity_mean               0
concave points_mean          0
symmetry_mean                0
fractal_dimension_mean       0
radius_se                    0
texture_se                   0
perimeter_se                 0
area_se                      0
smoothness_se                0
compactness_se               0
concavity_se                 0
concave points_se            0
symmetry_se                  0
fractal_dimension_se         0
radius_worst                 0
texture_worst                0
perimeter_worst              0
area_worst                   0
smoothness_worst             0
compactness_worst            0
concavity_worst              0
concave points_worst         0
symmetry_worst               0
fractal_dimension_worst      0
Unnamed: 32                569
dtype: int64
```

Confirmamos que 'Unnamed' é o único campo que possui valores nulos e que a quantidade de registros nulos deste campo é igual a quantidade de registros do dataset, ou seja a coluna não possui nenhum valor. A seguir faremos a exclusão para não ter que removê-lo a cada iteração.

In [ ]

```
# excluindo a coluna de nome 'Unnamed: 32'.
# axis=1 para indicar que está removendo a coluna.
# inplace=True indica que o dataframe original será alterado. O default (inplace=False) devolve uma cópia do d
if 'Unnamed: 32' in df.columns:
    df.drop('Unnamed: 32', axis=1, inplace=True)
```

In [ ]

```
#alterando o pandas para exibir todas as colunas. O default é exibir 20 colunas
pd.set_option('display.max_columns', None)
df.head()
```

```
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  fractal_dimension_mean  radius_se  texture_se  perimeter_se  \
0         0.2419                 0.07871     1.0950      0.9053         8.589
1         0.1812                 0.05667     0.5435      0.7339         3.398
2         0.2069                 0.05999     0.7456      0.7869         4.585
3         0.2597                 0.09744     0.4956      1.1560         3.445
4         0.1809                 0.05883     0.7572      0.7813         5.438
```

```
   area_se  smoothness_se  compactness_se  concavity_se  concave points_se  \
0   153.40       0.006399         0.04904       0.05373            0.01587
1    74.08       0.005225         0.01308       0.01860            0.01340
2    94.03       0.006150         0.04006       0.03832            0.02058
3    27.23       0.009110         0.07458       0.05661            0.01867
4    94.44       0.011490         0.02461       0.05688            0.01885

   symmetry_se  fractal_dimension_se  radius_worst  texture_worst  \
0      0.03003              0.006193
```

Dando prosseguimento à investigação dos dados exibimos os 5 primeiros registros. Em uma análise incial podemos ver que alguns campo estão com escalas bem diferente de outros. O nosso target (diagnosis) é um campo descritivo, porém nos primeiros 5 registros só é apresentado o valor M. Ao final da tabela notamos a ausência do campo que foi excluído no passo anterior.

In [ ]

```
# Verificando o domínio e a proporção de tipos de diagnósticos
df["diagnosis"].value_counts(normalize=True).mul(100).round(2).astype(str) + " %"
```

```
diagnosis
B    62.74 %
M    37.26 %
Name: proportion, dtype: object
```

Os únicos valores na coluna diagnosis são B (Benigno) e M (Maligno). Isto está aderente a documentação do dataset. A verificação do percentual de ocorrência de cada valor será importante para fazermos um stratify no momento que formos dividir os dados de treino e teste.

In [ ]

```
# Verificando os dados para analisar o quão diferente estão as escalas
df.describe()
```

```
                 id  radius_mean  texture_mean  perimeter_mean    area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000   569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033   654.889104
std    1.250206e+08     3.524049      4.301036       24.298981   351.914129
min    8.670000e+03     6.981000      9.710000       43.790000   143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000   420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000   551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000   782.700000
max    9.113205e+08    28.110000     39.280000      188.500000  2501.000000

       smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
min           0.052630          0.019380        0.000000             0.000000
25%           0.086370          0.064920        0.029560             0.020310
50%           0.095870          0.092630        0.061540             0.033500
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200

       symmetry_mean  fractal_dimension_mean   radius_se  texture_se  \
count     569.000000               569.000000  569.000000  569.000000
mean        0.181162                 0.062798    0.405172    1.216853
std         0.027414                 0.007060    0.277313    0.551648
min         0.106000                 0.049960    0.111500    0.360200
25%         0.161900                 0.057700    0.232400    0.833900
50%         0.179200                 0.061540    0.324200    1.108000
75%         0.195700                 0.066120    0.4789
```

Nota-se uma diferença nas escalas dos dados. Por exemplo area_mean varia entre 143.5 e 2501 enquanto fractal_dimension_mean varia entre 0.04996 e 0.09744

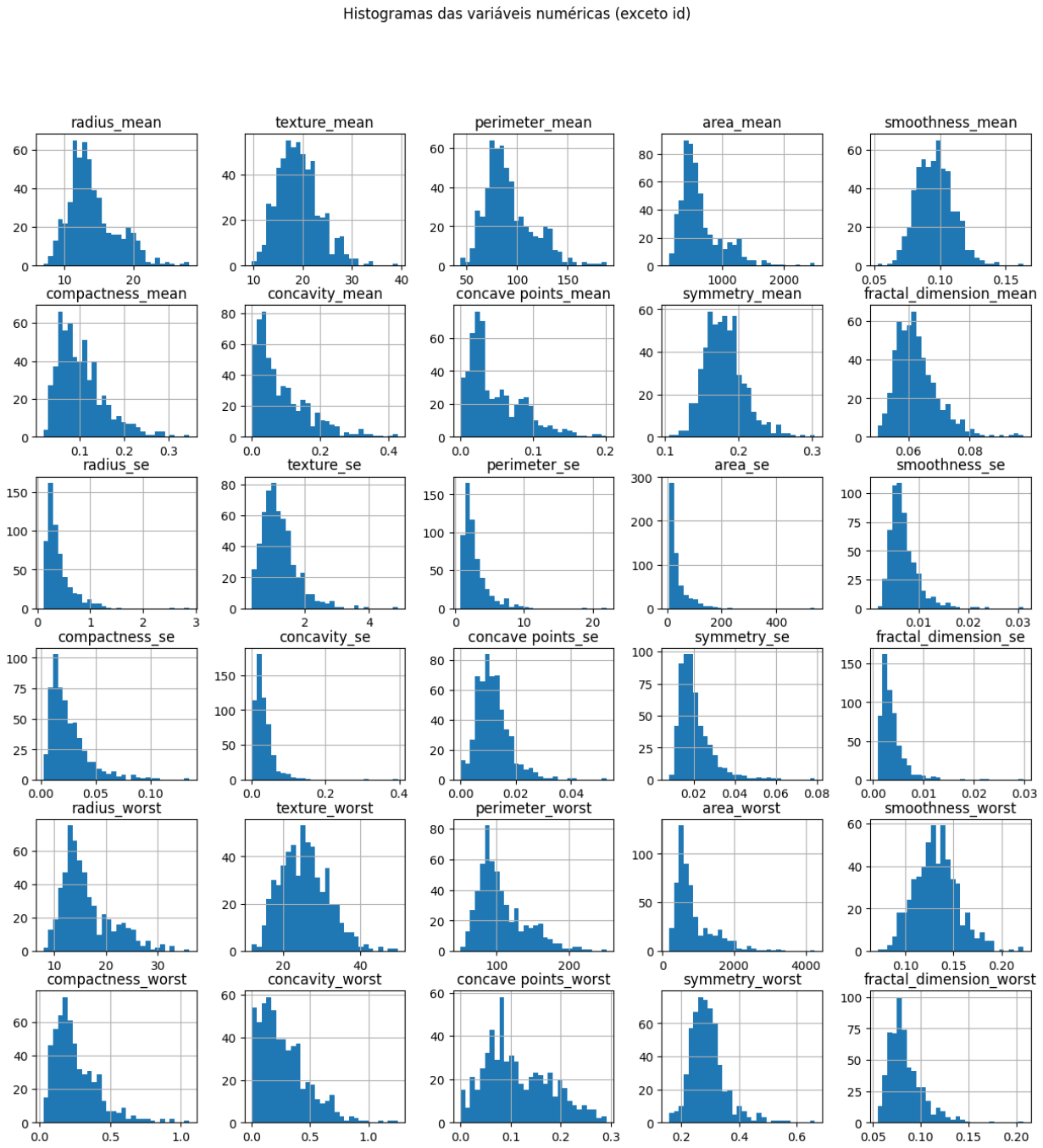
In [ ]

```
df
```

```
           id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0      842302         M        17.99         10.38          122.80     1001.0
1      842517         M        20.57         17.77          132.90     1326.0
2    84300903         M        19.69         21.25          130.00     1203.0
3    84348301         M        11.42         20.38           77.58      386.1
4    84358402         M        20.29         14.34          135.10     1297.0
..        ...       ...          ...           ...             ...        ...
564    926424         M        21.56         22.39          142.00     1479.0
565    926682         M        20.13         28.25          131.20     1261.0
566    926954         M        16.60         28.08          108.30      858.1
567    927241         M        20.60         29.33          140.10     1265.0
568     92751         B         7.76         24.54           47.92      181.0

     smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0            0.11840           0.27760         0.30010              0.14710
1            0.08474           0.07864         0.08690              0.07017
2            0.10960           0.15990         0.19740              0.12790
3            0.14250           0.28390         0.24140              0.10520
4            0.10030           0.13280         0.19800              0.10430
..               ...               ...             ...                  ...
564          0.11100           0.11590         0.24390              0.13890
565          0.09780           0.10340         0.14400              0.09791
566          0.08455           0.10230         0.09251              0.05302
567          0.11780           0.27700         0.35140              0.15200
568          0.05263           0.04362         0.00000              0.00000

     symmetry_mean  fractal_dimension_mean  radius_se  texture_se  \
0
```

In [ ]

```
# Plotar histogramas
df.drop(columns=["id"]).hist(figsize=(15,15), bins=30)
plt.suptitle("Histogramas das variáveis numéricas (exceto id)")
plt.show()
```



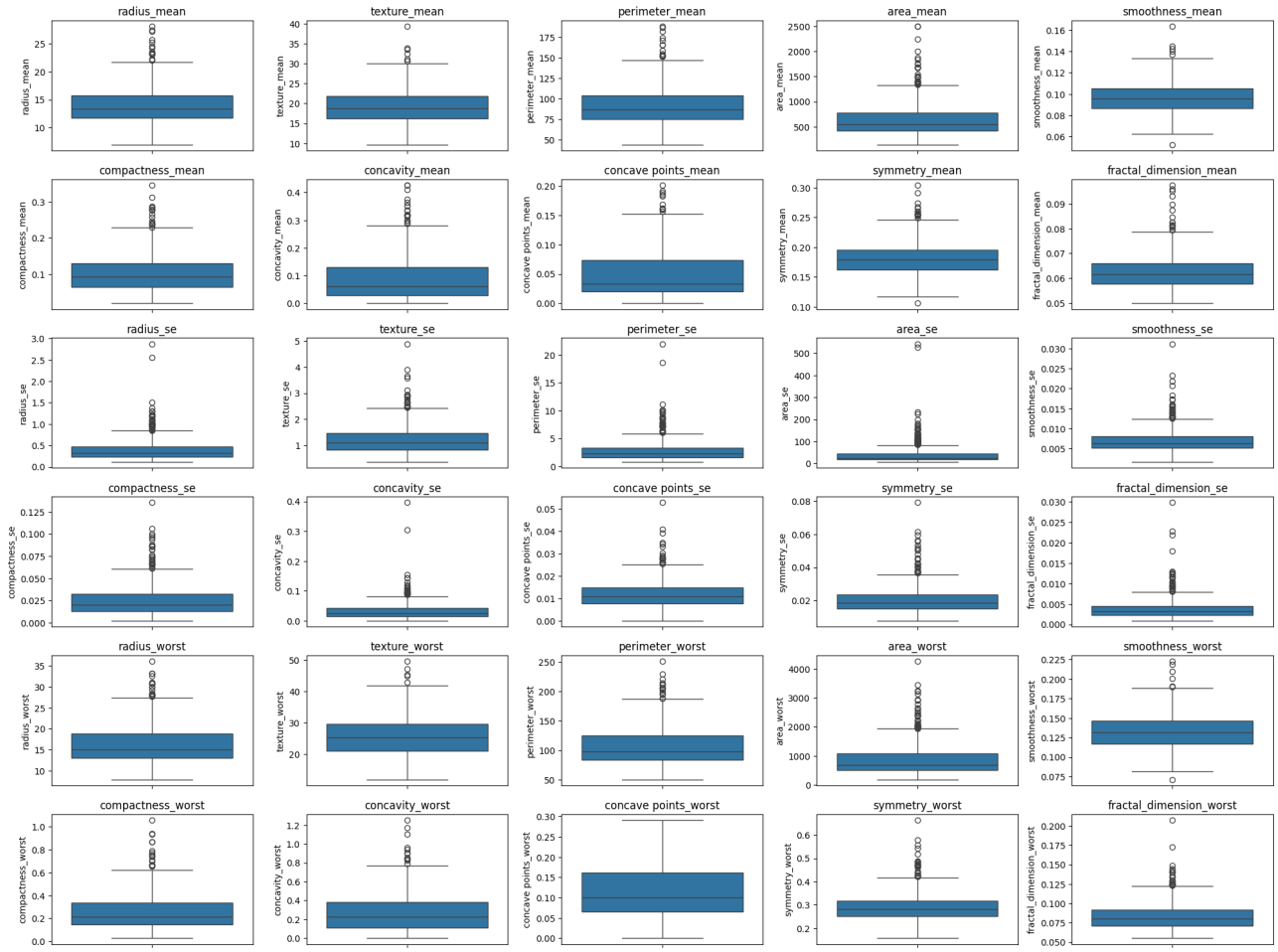
Algumas variáveis estão bem distribuídas aproximando-se de uma distribuição gaussiana (normal). Porém outras variáveis puxam o histograma para a esquerda apresentando poucas ocorrências a direita, o que pode indicar a presença de outliers.

Também podemos confirmar através do histograma que existem algumas variáveis em escalas muito diferentes, por exemplo area_mean e smoothness_mean.

In [ ]

```
# Verificar a presença de outliers nas features (exceto o campo id e a colunas Unnamed que está sobrando)
plt.figure(figsize=(20, 15))
for i, col in enumerate(df.drop(columns=["id","diagnosis"]).columns, 1):
    plt.subplot(6, 5, i)
    sns.boxplot(y=df[col])
    plt.title(col)

plt.tight_layout()
```

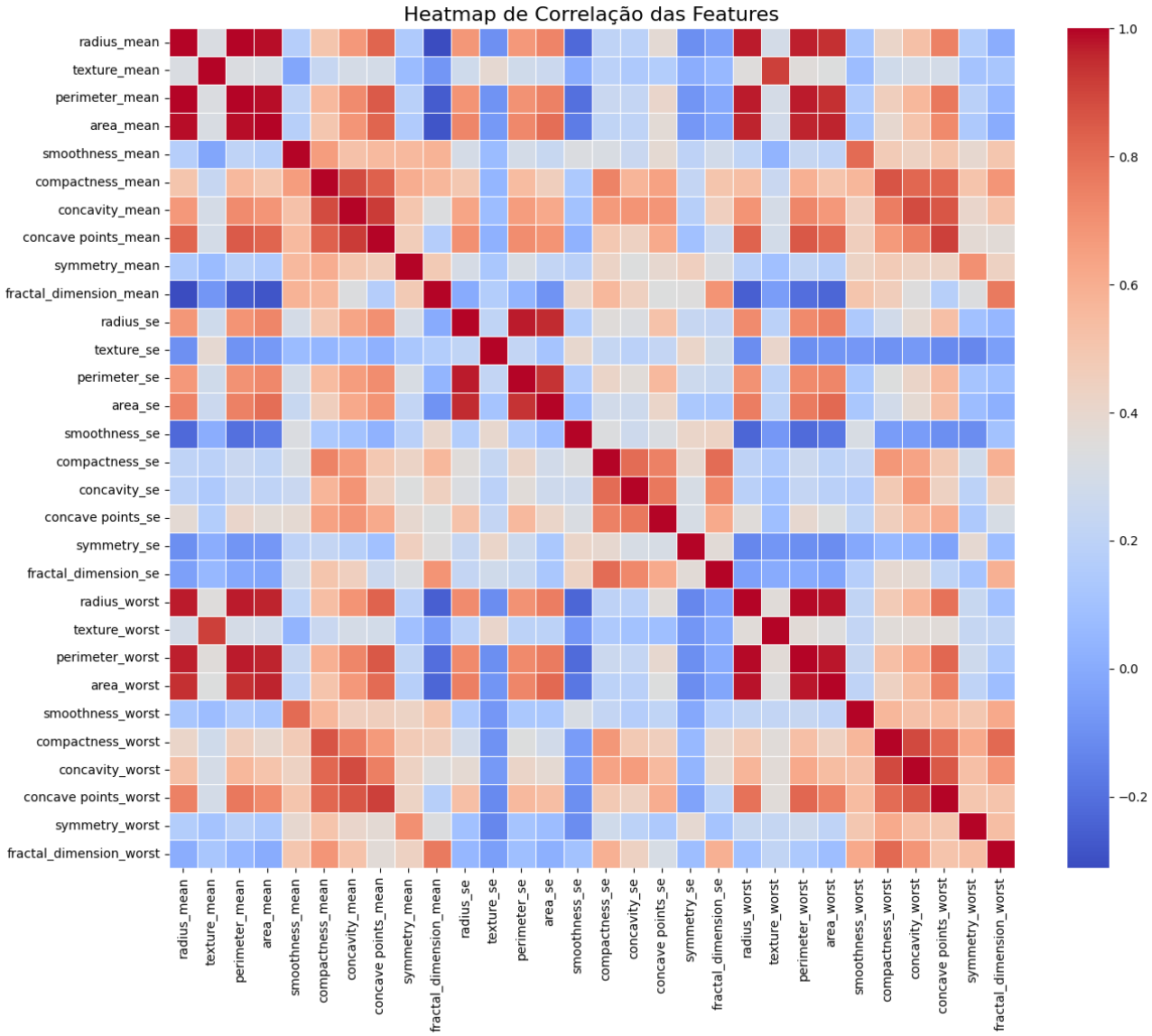
```
plt.show()
```



Podemos verificar nos gráficos boxplot que praticamente todos os campos, exceto concave_points_worst, possuem outliers. Porém no contexto de um dataset para predição de diagnóstico baseado em características de núcleos celulares, é esperado que existam valores extremos. Se excluirmos ou alterarmos estes valores podemos comprometer sinais importantes para a predição de um tumor maligno ou benigno. Desta forma vamos manter o dataset com os outliers.

In [ ]

```
# Plotar heatmap de correlação
plt.figure(figsize=(15,12))
sns.heatmap(df.drop(columns=["id","diagnosis"]).corr(), annot=False, cmap="coolwarm", linewidths=0.5)
plt.title("Heatmap de Correlação das Features", fontsize=16)
plt.show()
```



4. Preparação dos dados

In [ ]

```
# Separando as features do target
X = df.drop(columns=["id","diagnosis"])
y = df["diagnosis"]

# Tranformando a variável target categórica em uma variável numérica
y = LabelEncoder().fit_transform(y)
```

Criando pipeline para Random Forest Classifier

In [ ]

```
# Criar pipeline de pré-processamento
preprocessor_scaler = ColumnTransformer([
    ("num", StandardScaler(), X.columns) # deixa todas variáveis numéricas na mesma escala
])
```

In [ ]

```
# Separa base de treino e teste usando stratify para garantir que a proporção de benignos/malignos seja manti
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=df["diagnos
```

Aplicando o algoritmo Random Forest Classifier

In [ ]

```
random_forest_pipeline = Pipeline([
    ("preprocessorScaler", preprocessor_scaler),
    ("randomForest", RandomForestClassifier(
        n_estimators=100,      # número de árvores
        random_state=42,       # reprodutibilidade
        n_jobs=-1              # usa todos os núcleos disponíveis
    ))
])

# Treina o modelo
random_forest_pipeline.fit(X_train, y_train)

# Faz previsões
y_pred_random = random_forest_pipeline.predict(X_test)
y_prob_random = random_forest_pipeline.predict_proba(X_test)[:, 1]
```

In [ ]

```
def plotMatrix(test, pred):
    # Criar matriz de confusão
    cm = confusion_matrix(test, pred)

    # Plotar com seaborn para melhor visualização
    plt.figure(figsize=(6,4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.xlabel("Previsto")
    plt.ylabel("Real")
    plt.title("Matriz de Confusão")
    plt.show()
```

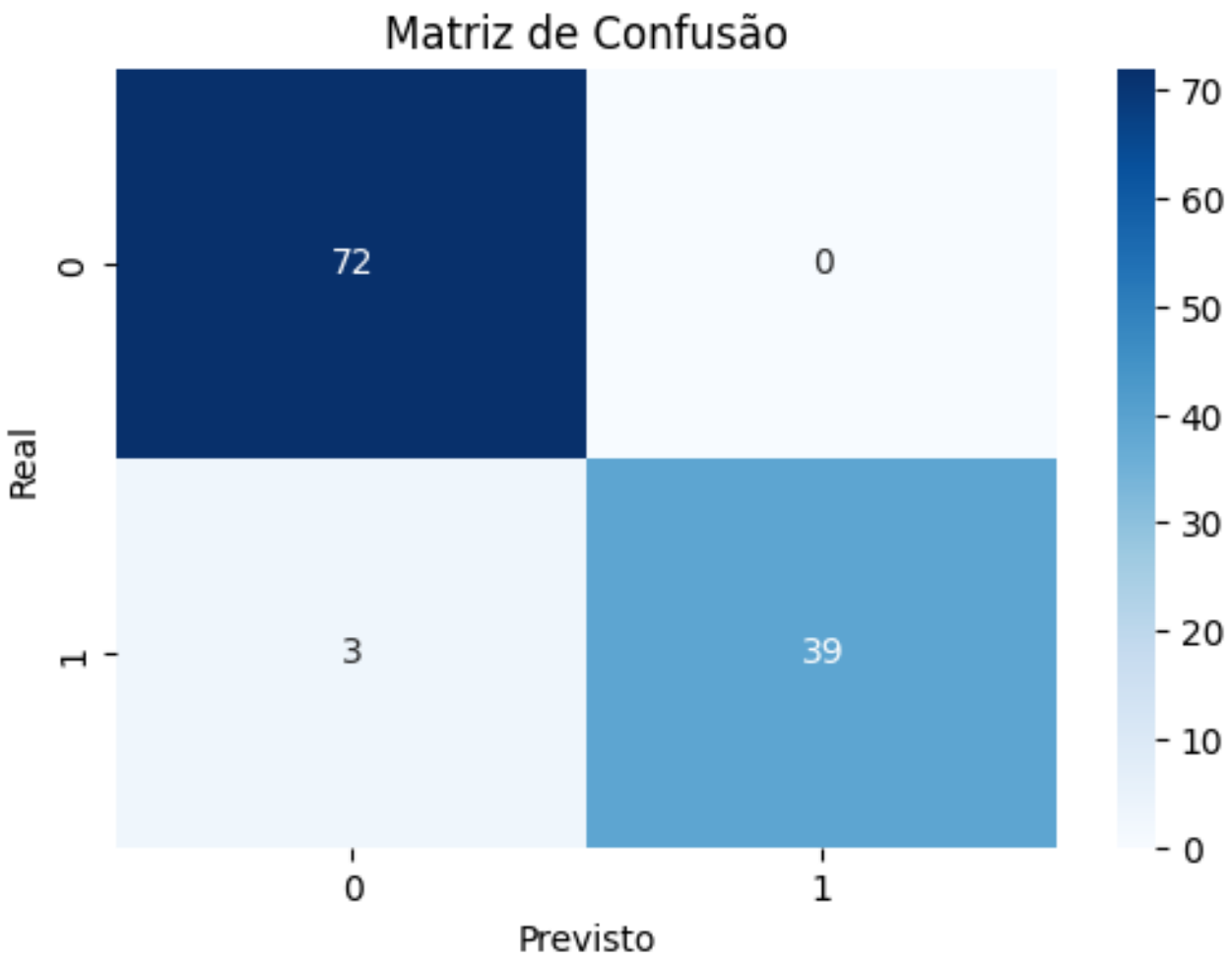
In [ ]

```
# Avaliar modelo
accuracy = accuracy_score(y_test, y_pred_random)
f1 = f1_score(y_test, y_pred_random)

print(f"Acurácia: {accuracy:.4f}")
print(f"F1-score: {f1:.4f}")

print("\nRelatório de classificação:")
print(classification_report(y_test, y_pred_random))
```

```
Acurácia: 0.9737
F1-score: 0.9630

Relatório de classificação:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        72
           1       1.00      0.93      0.96        42

    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

In [ ]

```
plotMatrix(y_test, y_pred_random)
```



Aplicando o algoritmo K-NN

In [ ]

```
#dados → StandardScaler → KNN → previsão
knn_pipeline = Pipeline([
     ("scaler", StandardScaler()),
     ("knn", KNeighborsClassifier(n_neighbors=3))
])
```

In [ ]

```
#treinar knn
knn_pipeline.fit(X_train, y_train)

y_pred_knn = knn_pipeline.predict(X_test)
y_prob_knn = knn_pipeline.predict_proba(X_test)[:, 1]
```

In [ ]

```
# avaliar
accuracy = accuracy_score(y_test, y_pred_knn)
```

In [ ]

```
f1 = f1_score(y_test, y_pred_knn, average="macro")
```

In [ ]

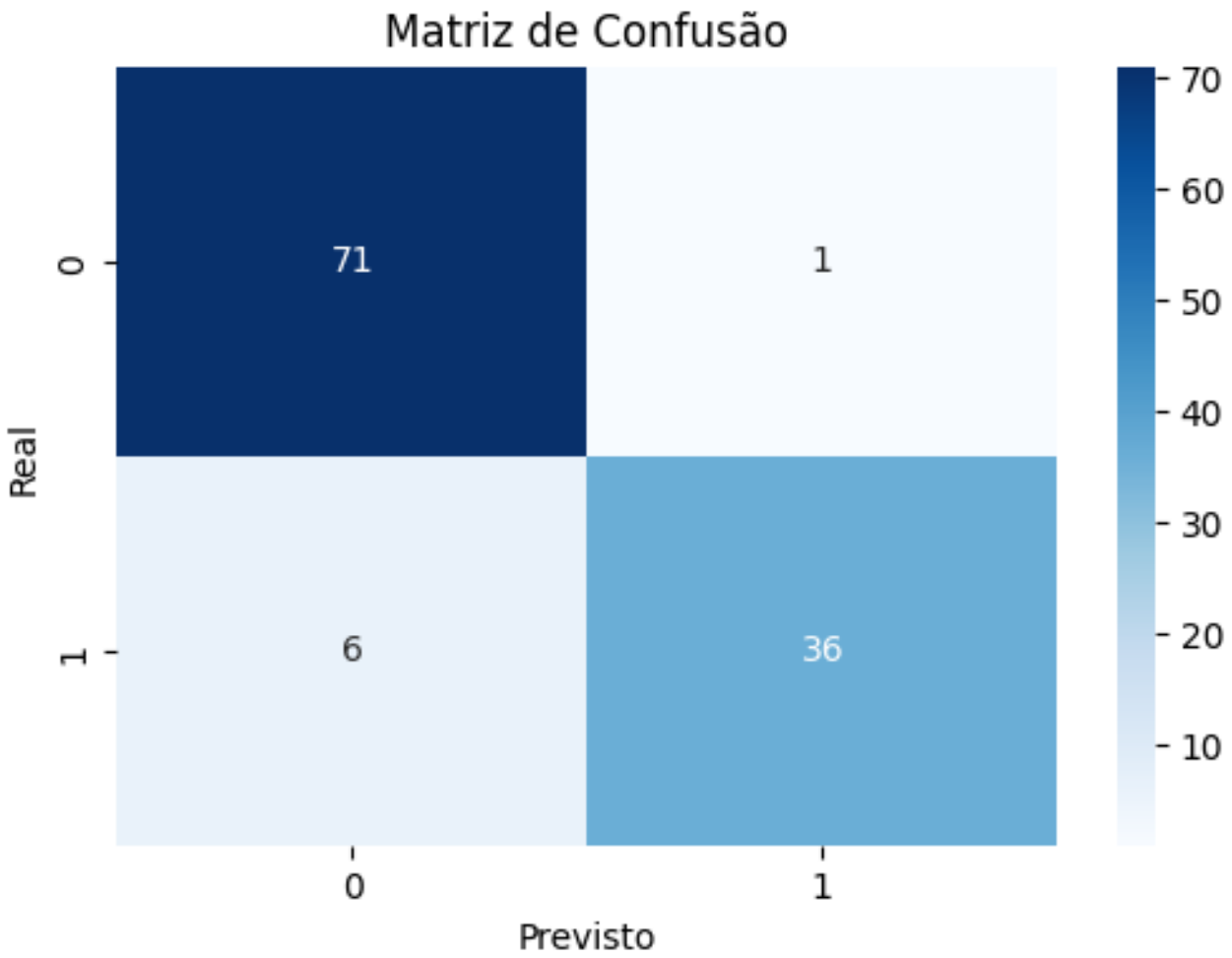
```
print(f"Acurácia: {accuracy:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
print("\nRelatório de classificação:")
print(classification_report(y_test, y_pred_knn))
```

```
Acurácia: 0.9386
F1-score: 0.9322

Relatório de classificação:
              precision    recall  f1-score   support

           0       0.92      0.99      0.95        72
           1       0.97      0.86      0.91        42

    accuracy                           0.94       114
   macro avg       0.95      0.92      0.93       114
weighted avg       0.94      0.94      0.94       114
```

In [ ]

```
plotMatrix(y_test, y_pred_knn)
```



6. Avaliação

Resultado obtidos

Os modelos foram avaliados no conjunto de teste (20% dos dados, com stratify para manter a proporção de classes). Os resultados observados nos dois modelos aplicados foram.

Métrica	Definição	Importância no Contexto
Acurácia	Proporção de predições corretas	Visão geral útil, mas insuficiente se isolada
Recall (Sensibilidade)	TP / (TP + FN) — malignos corretamente detectados	**MÉTRICA MAIS CRÍTICA**: FN = não detectar câncer = risco de vida
Precision	TP / (TP + FP)	Importante, mas secundária — FP gera apenas exames adicionais
F1-Score	Média harmônica Precision × Recall	Equilíbrio geral, bom para comparar modelos
ROC-AUC	Área sob a curva ROC	Avalia discriminação independente do threshold

In [ ]

```
def get_metrics(y_true, y_pred, y_prob, model_name):
    return {
        'Modelo': model_name,
        'Acurácia': accuracy_score(y_true, y_pred),
        'Recall (Maligno)': recall_score(y_true, y_pred),
        'Precision (Maligno)': precision_score(y_true, y_pred),
        'F1-Score': f1_score(y_true, y_pred),
        'ROC-AUC': roc_auc_score(y_true, y_prob)
    }

metrics_rf  = get_metrics(y_test, y_pred_random, y_prob_random, 'Random Forest')
metrics_knn = get_metrics(y_test, y_pred_knn,    y_prob_knn,    'KNN (k=3)')

df_metrics = pd.DataFrame([metrics_rf, metrics_knn]).set_index('Modelo')
df_metrics.style.format('{:.4f}').background_gradient(cmap='Blues', axis=0)
```

```
<pandas.io.formats.style.Styler at 0x7dbc17504230>
```

Em termos práticos, o Random Forest errou em menos de 3 casos a cada 100. O KNN errou em aproximadamente 5 a cada 100. Ambos os desempenhos são expressivos, mas o que importa mais do que o número geral é entender em que tipo de erro cada modelo falha.

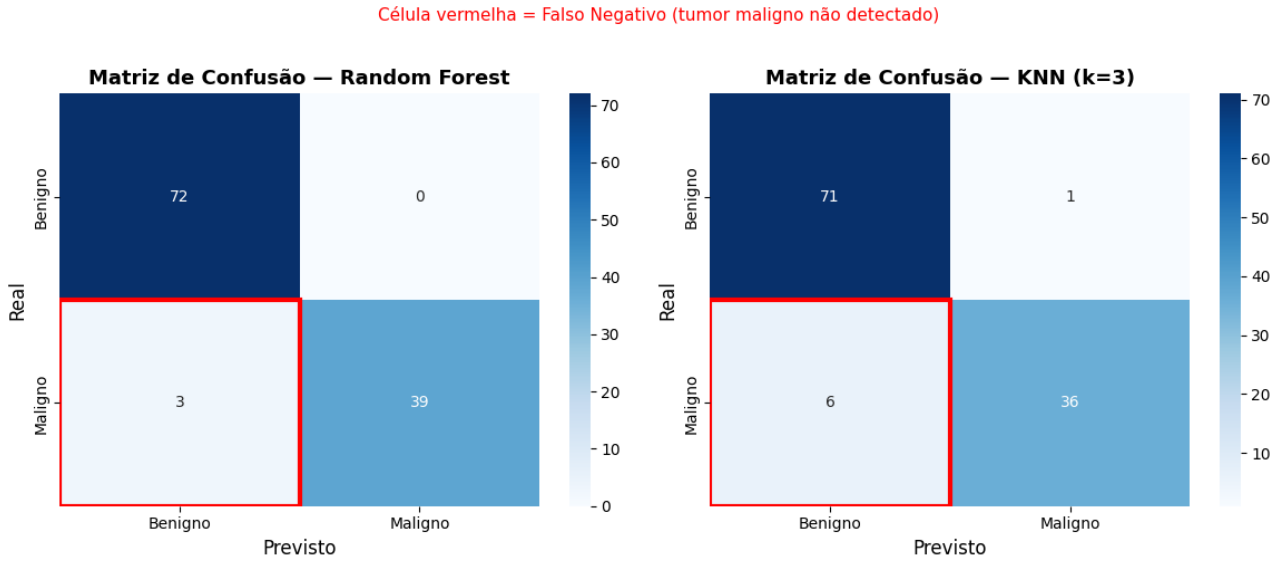
- Dizer que um tumor é benigno quando na verdade é maligno: este é o erro mais grave. O paciente pode não receber tratamento a tempo, com riscos sérios à saúde.
- Dizer que um tumor é maligno quando na verdade é benigno: este erro gera preocupação desnecessária e exames adicionais (indesejável, mas não coloca a vida em risco).

Para avaliar essa questão precisamos olhar as matrizes de confusão geradas e a métrica recall. O recall para a classe Maligno é a métrica prioritária já que um Falso Negativo (FN) significa não detectar um tumor maligno o que leva a consequências potencialmente fatais. Mesmo que isso aumente levemente os alarmes falsos (FP), o custo clínico de um FP é muito menor: resulta apenas em exames adicionais.

In [ ]

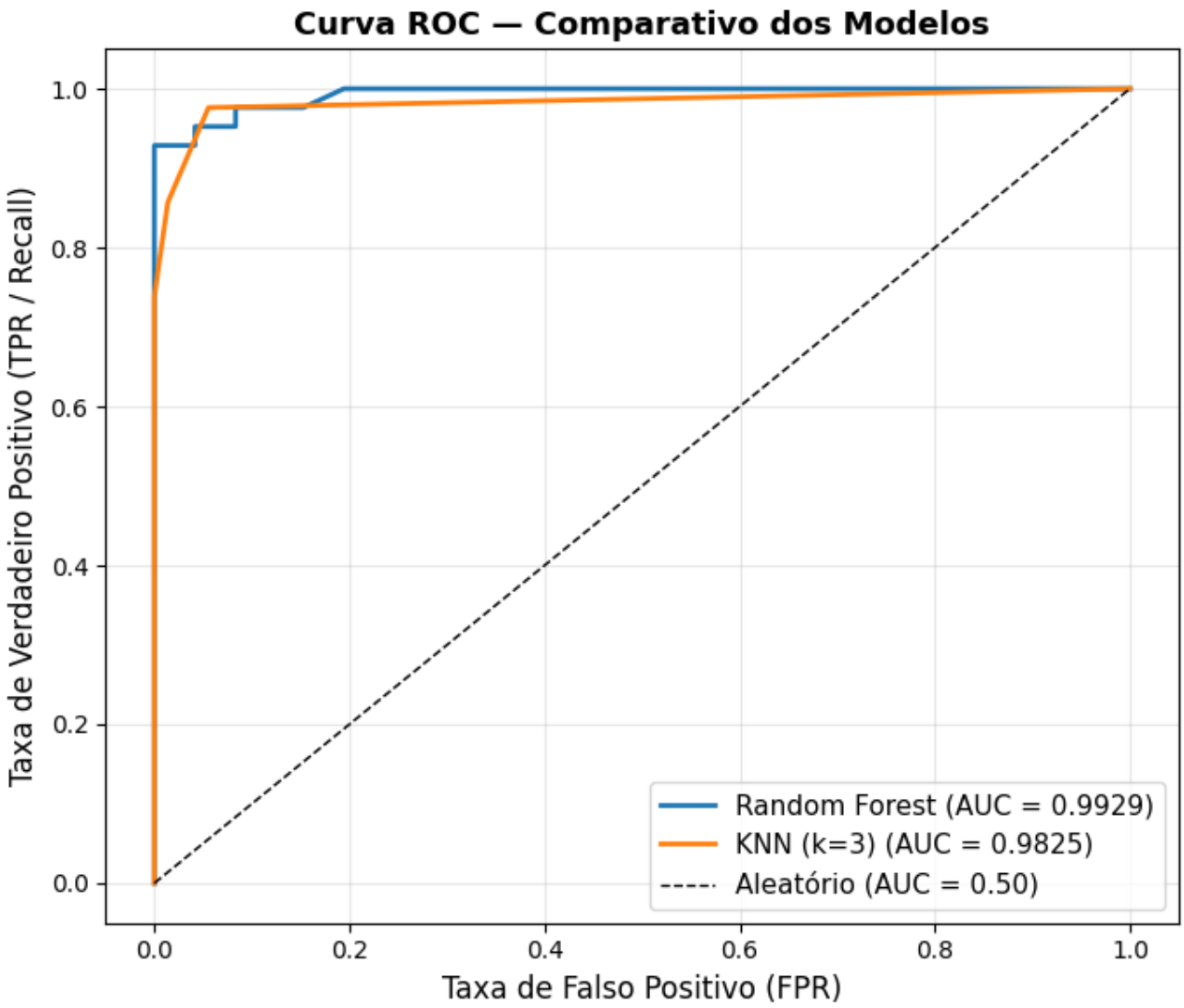
```
# ■■ Matrizes de Confusão ■■■
```

```
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Benigno', 'Maligno'],
                yticklabels=['Benigno', 'Maligno'])
    ax.set_xlabel('Previsto', fontsize=12)
    ax.set_ylabel('Real', fontsize=12)
    ax.set_title(f'Matriz de Confusão — {name}', fontsize=13, fontweight='bold')

    # Anotar FN (célula crítica: Real=Maligno, Previsto=Benigno)
    ax.add_patch(plt.Rectangle((0, 1), 1, 1, fill=False,
                               edgecolor='red', lw=3, label='FN (crítico)'))

plt.suptitle('Célula vermelha = Falso Negativo (tumor maligno não detectado)',
             fontsize=11, color='red', y=1.02)
plt.tight_layout()
plt.show()
```



In [ ]

```
# ■■ Curva ROC ■■■
```



Dessa forma comparando as matrizes de confusão e a curva ROC observamos uma pequena diferença entre os modelos. Essa diferença, embora pareça pequena, é clinicamente relevante. Em um hospital que analisa mil biópsias por mês, a diferença entre ~4% e ~6% de casos malignos não detectados representa dezenas de pacientes. Por isso o Random Forest é o modelo recomendado para este contexto.

##Interpretabilidade: Feature Importance e SHAP

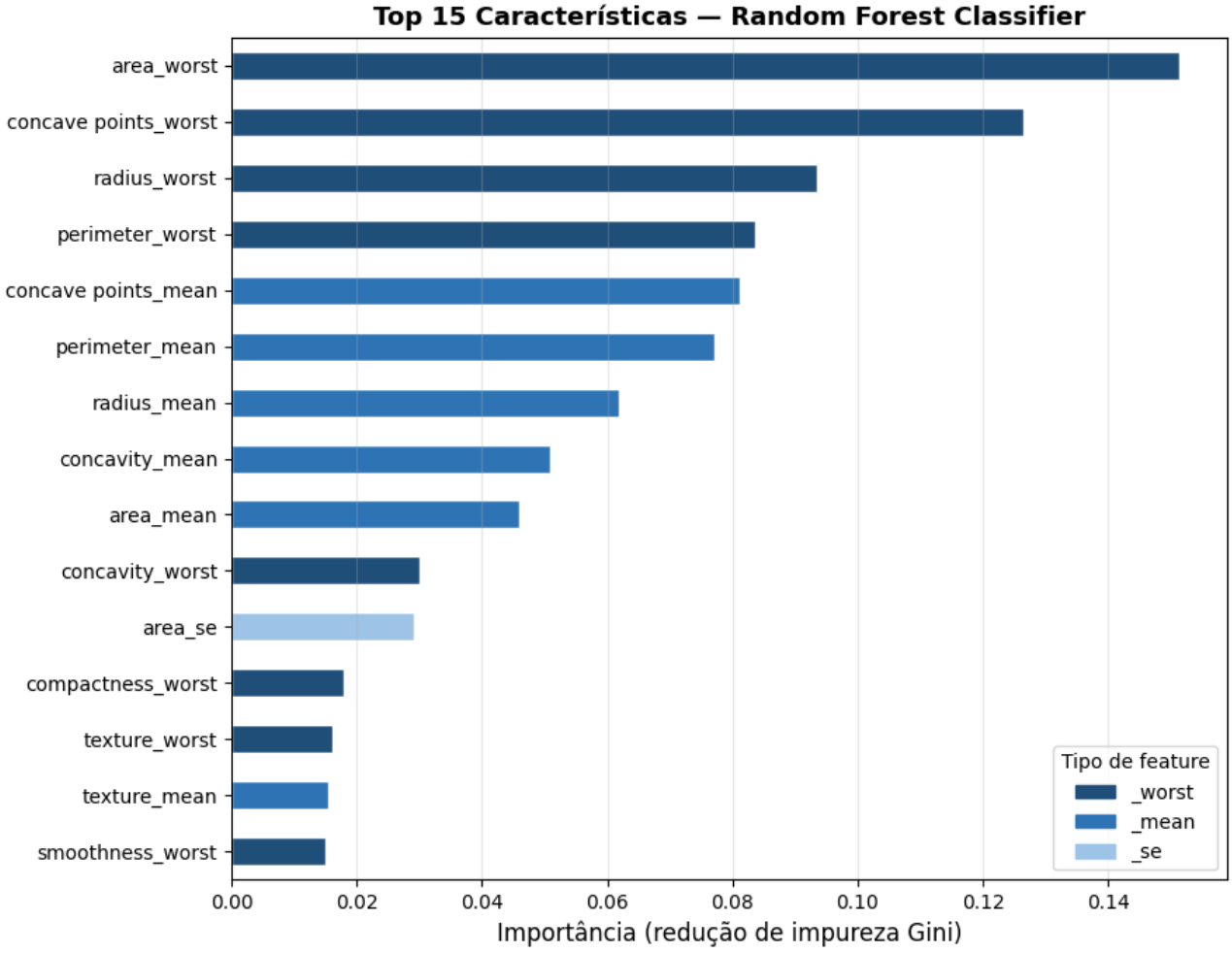
Feature Importance — Random Forest

O Random Forest fornece nativamente `feature_importances_`, que mede a redução média de impureza (Gini) atribuída a cada feature ao longo de todas as árvores.

In [ ]

```
rf_model = random_forest_pipeline.named_steps['randomForest']
importances = pd.Series(rf_model.feature_importances_, index=X.columns)
importances_sorted = importances.sort_values(ascending=True)

# Pegando as 15 maiores
top15 = importances_sorted.tail(15)

fig, ax = plt.subplots(figsize=(9, 7))
colors = ['#1f4e79' if '_worst' in f else '#2e74b5' if '_mean' in f else '#9dc3e6'
          for f in top15.index]
```

```
top15.plot(kind='barh', ax=ax, color=colors, edgecolor='white')

ax.set_xlabel('Importância (redução de impureza Gini)', fontsize=12)
ax.set_title('Top 15 Características — Random Forest Classifier', fontsize=13, fontweight='bold')

patches = [
    mpatches.Patch(color='#1f4e79', label='_worst'),
    mpatches.Patch(color='#2e74b5', label='_mean'),
    mpatches.Patch(color='#9dc3e6', label='_se'),
]
ax.legend(handles=patches, title='Tipo de feature', fontsize=10)
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

#Restringindo as top 10 características
print('\nTop 10 Características por importância:')
print(importances_sorted.tail(10).sort_values(ascending=False).to_frame('Importância').round(4))
```




```
Top 10 Características por importância:
                      Importância
area_worst                 0.1514
concave points_worst       0.1265
radius_worst               0.0935
perimeter_worst            0.0836
concave points_mean        0.0811
perimeter_mean             0.0771
radius_mean                0.0620
concavity_mean             0.0508
area_mean                  0.0459
```

```
concavity_worst            0.0300
```

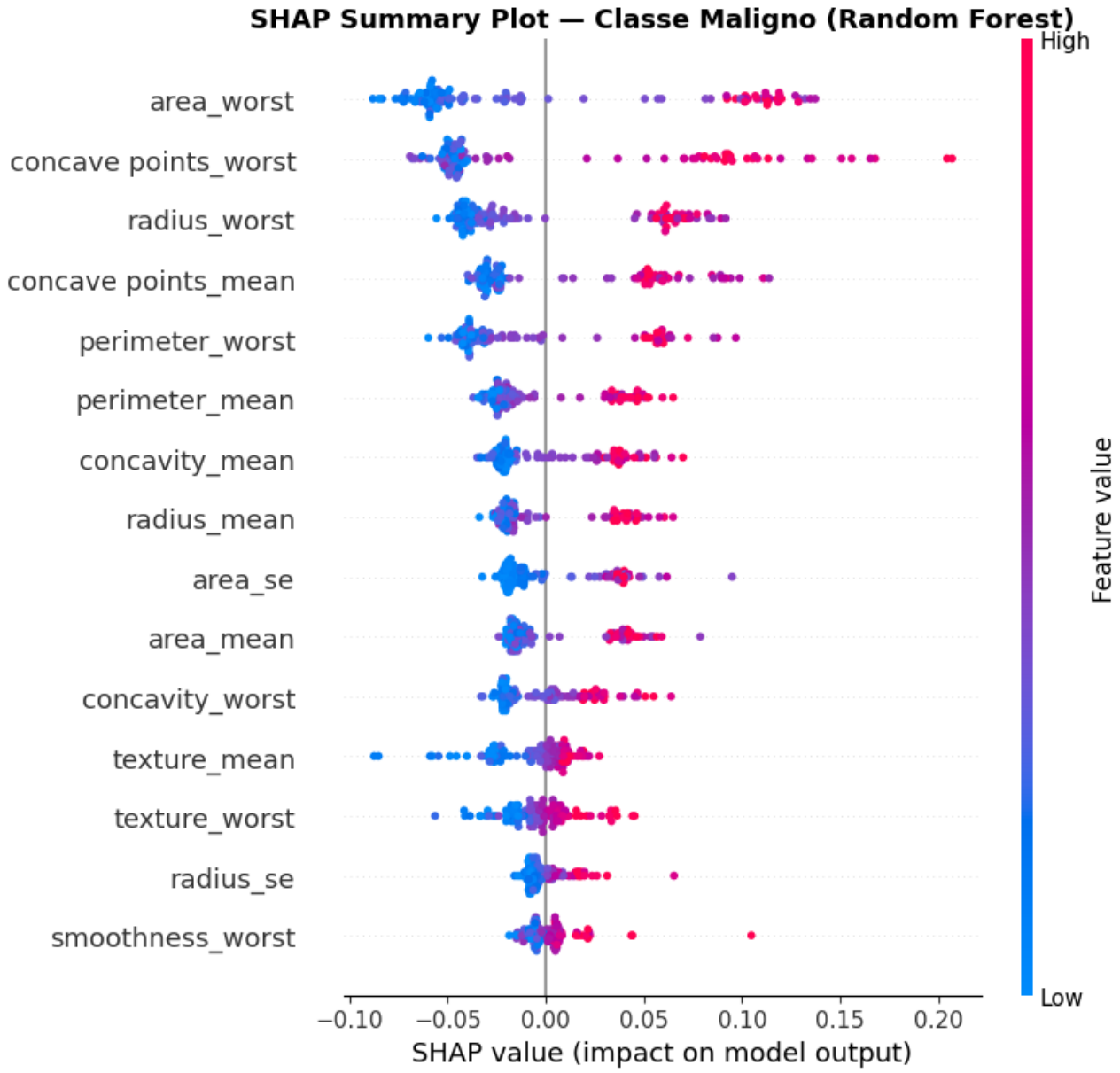
###SHAP — SHapley Additive exPlanations

O SHAP é baseado na teoria dos jogos cooperativos (valores de Shapley) e oferece vantagens sobre a Feature Importance como direção do efeito e explicação por amostra. Usamos `TreeExplainer` por ser otimizado para modelos baseados em árvores.

In [ ]

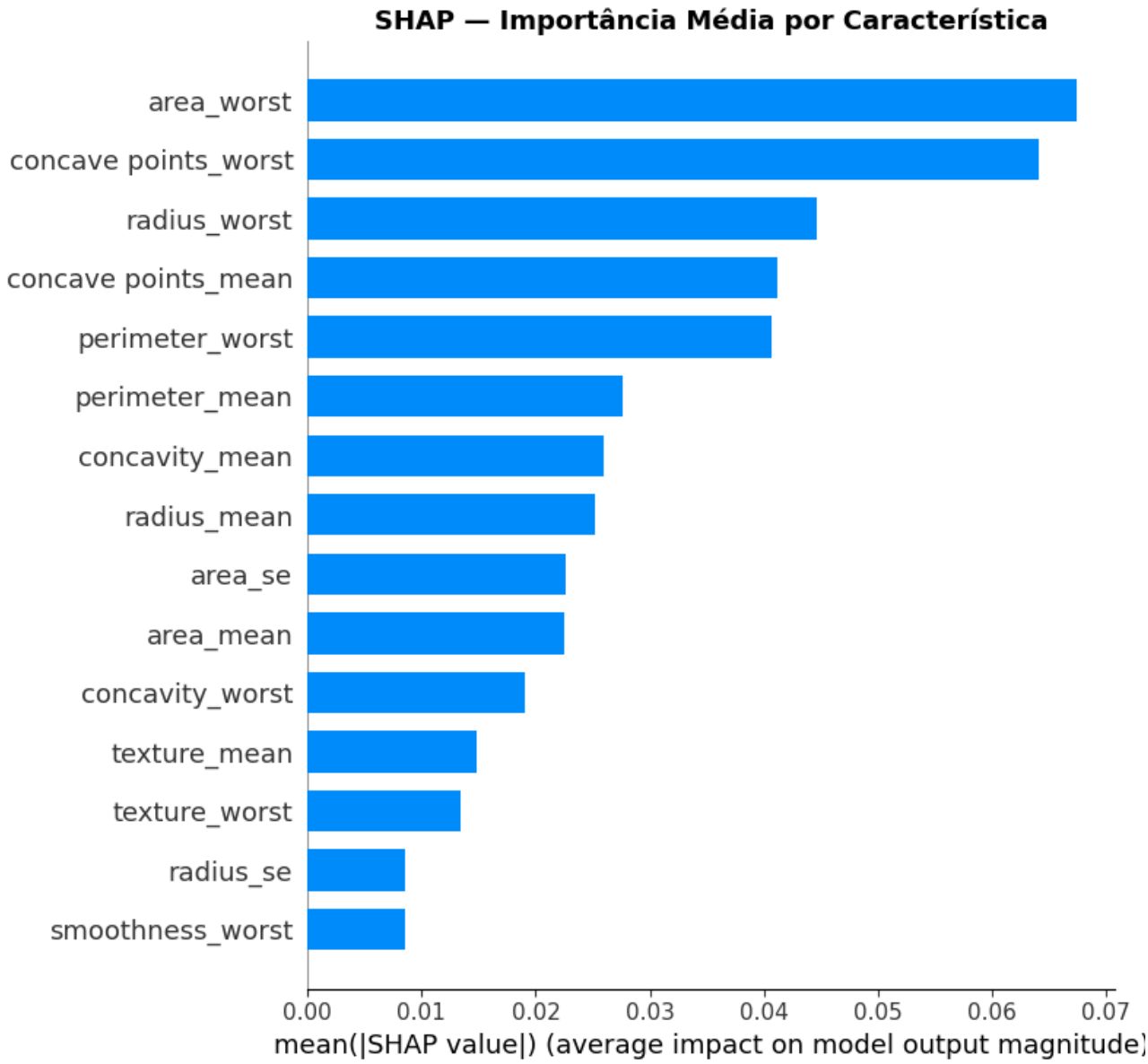
```
# Preparação do SHAP
# Extraindo dados escalados do pipeline
X_test_scaled = random_forest_pipeline.named_steps['preprocessorScaler'].transform(X_test)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

# TreeExplainer — eficiente para Random Forest
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test_scaled_df)

# shap_values[1] = valores SHAP para a classe Maligno (1)
if isinstance(shap_values, list):
    shap_maligno = shap_values[1]
else:
    shap_maligno = shap_values[:, :, 1]

print(f'Forma dos valores SHAP: {shap_maligno.shape}')
print('SHAP calculado para a classe Maligno (1)')

# Cada ponto = 1 paciente
# Posição horizontal = contribuição para predição de Maligno
# Cor = valor da feature (vermelho=alto, azul=baixo)

plt.figure(figsize=(10, 8))
shap.summary_plot(
    shap_maligno,
    X_test_scaled_df,
    feature_names=X.columns.tolist(),
    plot_type='dot',
    max_display=15,
    show=False
)
plt.title('SHAP Summary Plot — Classe Maligno (Random Forest)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
Forma dos valores SHAP: (114, 30)
SHAP calculado para a classe Maligno (1)
```



In [ ]

```
# SHAP Importância média global
plt.figure(figsize=(9, 7))
shap.summary_plot(
    shap_maligno,
    X_test_scaled_df,
    feature_names=X.columns.tolist(),
    plot_type='bar',
    max_display=15,
    show=False
)
plt.title('SHAP — Importância Média por Característica', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

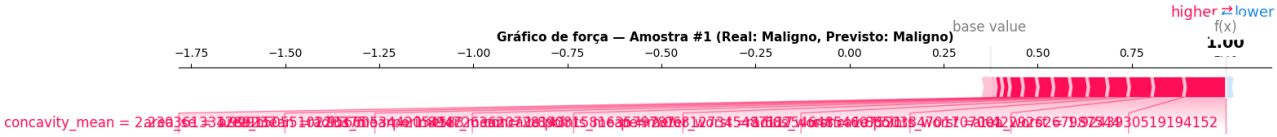


In [ ]

```
#SHAP Explicação Individual
# Escolhemos o primeiro caso maligno do conjunto de teste para análise

idx_maligno = np.where(y_test == 1)[0][0]  # primeiro caso maligno
pred_label = 'Maligno' if y_pred_random[idx_maligno] == 1 else 'Benigno'
real_label = 'Maligno' if y_test[idx_maligno] == 1 else 'Benigno'

print(f'Amostra #{idx_maligno} | Real: {real_label} | Previsto: {pred_label}')
print('\nForce Plot — Características em vermelho empurram para Maligno, azul para Benigno:')

shap.initjs()
shap.force_plot(
    explainer.expected_value[1],
    shap_maligno[idx_maligno],
    X_test_scaled_df.iloc[idx_maligno],
    feature_names=X.columns.tolist(),
    matplotlib=True,
    show=False,
    figsize=(16, 3)
)
plt.title(f'Gráfico de força — Amostra #{idx_maligno} (Real: {real_label}, Previsto: {pred_label})',
```

```
          fontsize=11, fontweight='bold')
plt.tight_layout()
plt.show()
```

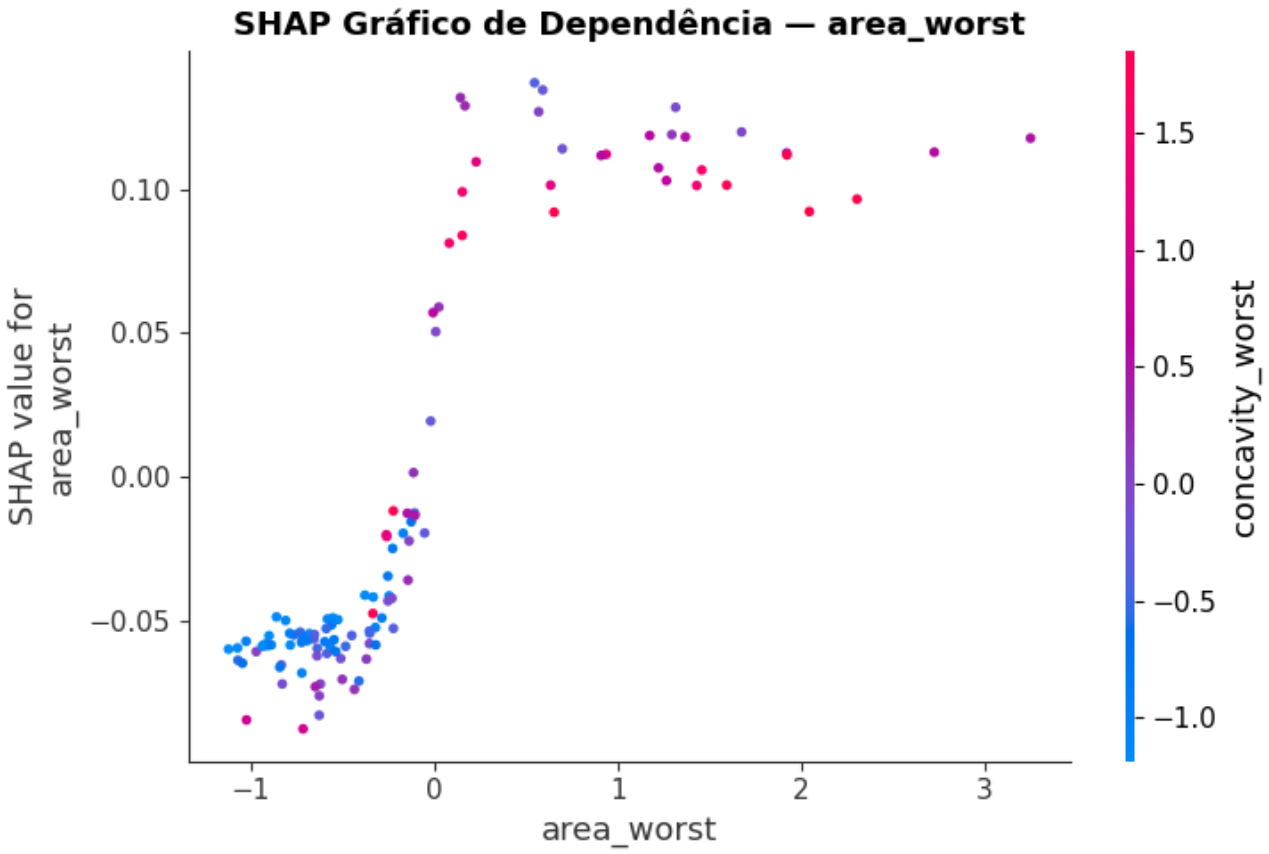
```
Amostra #1 | Real: Maligno | Previsto: Maligno

Force Plot — Características em vermelho empurram para Maligno, azul para Benigno:
<IPython.core.display.HTML object>
```



In [ ]

```
# SHAP de característica mais importante
# Mostra relação entre valor da feature e seu impacto SHAP
# A cor representa a interação com outra feature (escolhida automaticamente pelo SHAP)

top_feature = importances.idxmax()
print(f'Feature mais importante: {top_feature}')

plt.figure(figsize=(8, 5))
shap.dependence_plot(
    top_feature,
    shap_maligno,
    X_test_scaled_df,
    feature_names=X.columns.tolist(),
    show=False
)
plt.title(f'SHAP Gráfico de Dependência — {top_feature}', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
Feature mais importante: area_worst
<Figure size 800x500 with 0 Axes>
```



De acordo com os dados obtidos e gráficos tanto por meio da técnica de Feature Importance e SHAP observamos que as duas são convergentes demonstrando que as características com `_worst` dominam em ambas as análises.

- Características extremas de núcleos celulares (valor máximo observado) são mais diagnósticas do que médias, pois uma única célula atípica pode sinalizar malignidade.
- `concave_points_worst`, `perimeter_worst` e `radius_worst` são consistentemente as mais relevantes.

O SHAP adiciona direção: pontos vermelhos (valores altos) à direita no Summary Plot confirmam que valores altos dessas features aumentam a probabilidade de classificação como Maligno.

Com isso, temos candidatas à redução de dimensionalidade por apresentarem valores baixos (ex: `texture_se`, `fractal_dimension_mean`). Essas características poderiam ser removidas sem impacto significativo no desempenho, simplificando o modelo.

###Uso na prática

Os resultados mostram que o modelo tem desempenho suficiente para ser útil em um contexto clínico real, porém não como substituto do diagnóstico médico e sim complementar/auxiliar. O modelo funciona como um assistente experiente, que analisou várias biópsias e aprendeu a reconhecer padrões, sendo assim ele pode sinalizar riscos, priorizar casos urgentes e oferecer uma segunda opinião, mas a decisão final sempre cabe ao médico.

Como potencial de uso desse modelo temos

Aplicação	Descrição

Triagem	Sinalizar automaticamente casos com maior probabilidade de malignidade, priorizando-os na fila de análise médica
Opinião computacional	Em casos limítrofes, fornecer perspectiva adicional baseada em padrões estatísticos
Redução de variabilidade	Diagnósticos por análise visual sofrem variabilidade inter-observador; o modelo padroniza

Apesar dos resultados interessantes, ainda há etapas importantes que precisariam ser cumpridas antes de qualquer implantação:

- Validação com novos pacientes: o modelo foi avaliado com dados do mesmo estudo que gerou o conjunto de treinamento. Antes do uso real, seria necessário testá-lo com biópsias de outros hospitais, equipamentos e populações para verificar se o desempenho se mantém.
- Incorporação de mais informações clínicas: atualmente, o modelo considera apenas as características celulares. Integrar informações como idade da paciente, histórico familiar e resultado de mamografia tornaria as previsões mais completas e confiáveis.
- Aprovação regulatória: no Brasil, qualquer software com finalidade diagnóstica precisa de aprovação da ANVISA antes de ser utilizado em ambiente hospitalar.

O modelo vem como mais uma ferramenta de apoio para auxiliar os médicos no seu dia dia apoiado por novas tecnologias como o uso de inteligência artificial.